Dustoff Reset - Technical Capabilities

Version: 0.1.0
Platform: macOS (Windows/Linux planned)
Architecture: Tauri 2.x (Rust backend + React/TypeScript frontend)

Executive Summary

Dustoff Reset is a **cognitive capacity management system** that operates as a desktop overlay application. It monitors user behavior in real-time, applies biologically-grounded algorithms to track focus "bandwidth," and intervenes when cognitive capacity drops to prevent unproductive work sessions.

Table of Contents

1. System Architecture
2. Real-Time App Monitoring
3. Cognitive Bandwidth Engine
4. Daily Calibration System
5. Intervention System
6. Flow State Detection
7. Session Modes
8. Parking Lot (Thought Capture)
9. Reset Rituals
10. Gamification & Virality
11. Data Persistence
12. Window Management
13. Permission System
14. Cross-Platform Support

---

1. System Architecture

1.1 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + TypeScript	UI components, state management
Styling	Tailwind CSS	Dark glassmorphism design system
Desktop Runtime	Tauri 2.x	Native window, system APIs
Backend	Rust	Performance-critical logic, OS integration
Database	SQLite (via Rust)	Local data persistence
IPC	Tauri Commands	Type-safe frontend ↔ backend communication

1.2 Application Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                          FRONTEND (React)                        │
│                                                                  │
│   ┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────────────┐  │
│   │   HUD   │   │ Panels  │   │ Modals  │   │ Interventions   │  │
│   └─────────┘   └─────────┘   └─────────┘   └─────────────────┘  │
│        │             │             │                 │          │
│        └─────────────┴─────────────┴─────────────────┘          │
│                                │                                 │
│                                │                                 │
│                        ┌──────────────┐                          │
│                        │ Tauri Bridge │                          │
│                        └──────────────┘                          │
└──────────────────────────────────────────────────────────────────┘
                                │
                                │ IPC (invoke)
┌──────────────────────────────────────────────────────────────────┐
│                        BACKEND (Rust)                            │
│                                                                  │
│   ┌──────────┐   ┌──────────────┐   ┌──────────────┐   ┌────────┐ │
│   │ Commands │   │   Telemetry  │   │   Storage    │   │ Badges │ │
│   │          │   │   Platform   │   │   Database   │   │ Streaks│ │
│   └──────────┘   └──────────────┘   └──────────────┘   └────────┘ │
│                          │                                       │
│                  ┌──────────────────┐                            │
│                  │  OS-Level APIs   │                            │
│                  │  (NSWorkspace,   │                            │
│                  │   AppleScript)   │                            │
│                  └──────────────────┘                            │
└──────────────────────────────────────────────────────────────────┘
```

---

2. Real-Time App Monitoring

2.1 Frontmost Application Detection (macOS)

The system uses **Objective-C runtime bindings** to detect the currently focused application:

```
// Uses NSWorkspace API directly
let workspace: *mut Object = msg_send![class!(NSWorkspace),
sharedWorkspace];
let frontmost_app: *mut Object = msg_send![workspace,
frontmostApplication];
let app_name: String = msg_send![frontmost_app, localizedName];
let bundle_id: String = msg_send![frontmost_app, bundleIdentifier];
```

Data Captured:

- Application name (e.g., "Google Chrome")
- Bundle identifier (e.g., "com.google.Chrome")
- Window title (via AppleScript)

- Active since timestamp

2.2 Window Title Detection

For browsers and multi-window apps, window titles are captured via AppleScript:

```
tell application "System Events"
    tell process "Google Chrome"
        if exists (window 1) then
            return name of window 1
        end if
    end tell
end tell
```

Use Cases:

- Detect specific browser tabs (YouTube, Twitter, etc.)
- Identify document names in productivity apps
- Track context switches within the same application

2.3 Idle Time Detection

System idle time is detected via IOKit:

```
// Uses ioreg to get HIDIdleTime
Command::new("ioreg")
    .args(["-c", "IOHIDSystem", "-d", "4"])
    .output()
```

Use Cases:

- Pause bandwidth decay during idle
- Detect user returning from break
- Inform reset ritual recommendations

2.4 Polling Frequency

Event Type	Polling Interval
Frontmost app	1 second
Window title	1 second
Idle time	5 seconds

---

3. Cognitive Bandwidth Engine

3.1 Core Concept

Bandwidth is a 0-100 score representing the user's current cognitive capacity:

Range	State	Meaning
75-100	Optimal	Ready for deep work, flow eligible
60-74	Normal	Functional, minor friction
50-59	Warning	Friction intervention triggered
0-49	Critical	Focus-slipping intervention triggered

3.2 Bandwidth Events

Penalties (reduce bandwidth):

Event	Penalty	Trigger
Friction (manual)	-5	User reports feeling stuck
Focus-slipping (manual)	-10	User reports mind wandering
Non-whitelisted app	-12	Opens blocked app
Non-whitelisted app (repeat)	-6	Same app within 2 min
Tab switch	-2	Browser tab change
Tab burst (>5 in 60s)	-5	Frantic switching
App switch	-4	Application change
App burst (>3 in 60s)	-6	Rapid app switching

Gains (increase bandwidth):

Event	Gain	Trigger
Sustained focus	+1/min	No distractions for 1 minute
Flow celebration	+5	Entering flow state
Flow streak	+1/min	Each minute in flow
Breath Reset	+5	Complete 2-min ritual
Walk Reset	+7.5	Complete 5-min ritual
Dump Reset	+6	Complete parking lot dump

3.3 Mode-Specific Penalty Weights

Different session modes apply penalty multipliers:

Mode	Non-Whitelisted App	Tab Switch	App Switch
Zen	0.75x	0.5x	0.5x

Mode	Non-Whitelisted App	Tab Switch	App Switch
Flow	1.0x	1.0x	1.0x
Legend	1.5x	1.25x	1.25x

---

4. Daily Calibration System

4.1 Calibration Ceremony

Before each workday, users complete a **5-minute calibration** that establishes their baseline bandwidth:

Step 1: Sleep Assessment (40 points max)

- Sleep hours (25 pts): Optimal 7-9 hours
- Sleep quality (15 pts): User rating 1-10

Step 2: Emotional State (40 points max)

- Emotional residue (20 pts): Lingering stress level 1-10
- Current state (20 pts): Energized, Focused, Calm, Tired, Anxious, Scattered

Step 3: Distraction Awareness (20 points max)

- Identify potential distractions (0-6 items)
- Fewer = higher score (counterintuitive: awareness is the goal)

4.2 Calibration Expiry

Calibrations expire at **5:00 AM local time** (workday boundary):

```
if (currentHour < 5) {
    // Use yesterday's date
    calibrationDate = yesterday
}
```

---

5. Intervention System

5.1 Intervention Types

Type	Trigger	Purpose
Friction	Bandwidth 50-59	Warning: capacity declining
Focus-Slipping	Bandwidth <50	Critical: immediate action needed
Non-Whitelisted App	Opens blocked app	Immediate violation
Tab Burst	>5 tabs in 60s	Frantic behavior detected

5.2 Mode-Specific Responses

Zen Mode:

- Gentle, calming tone
- Emerald green theme
- Auto-dismisses after 10 seconds
- Encourages self-compassion

Flow Mode:

- Focused, practical tone
- Cyan blue theme
- Auto-dismisses after 10 seconds
- Emphasizes flow preservation

Legend Mode:

- Aggressive, confrontational tone
- Red theme with pulsing glow
- **No auto-dismiss** (requires action)
- Escalating consequences

5.3 Intervention Actions

Flow Mode - Delay Gate:

1. Intervention appears with 10-second countdown
2. User can "Minimize App" → App is minimized, returns to work
3. User can "Continue Anyway" → Proceeds but takes bandwidth penalty
4. Timer expires → Auto-returns to work

Legend Mode - Block Screen:

1. Full-screen red overlay blocks work
2. User must acknowledge and return
3. Bandwidth penalty applied
4. Tab/app can be force-closed

---

6. Flow State Detection

6.1 Entry Conditions (ALL must be true)

Condition	Threshold
Sustained focus	≥12 minutes
No context switches	12 minutes
No interventions	12 minutes

Condition	Threshold
Bandwidth	≥75

6.2 Exit Conditions (ANY triggers exit)

- Any context switch (tab or app)
- Any intervention triggered
- Bandwidth drops below 75
- Session paused/ended

6.3 Flow Celebration

When flow is detected:

1. Celebration overlay appears with particle effects
2. +5 bandwidth bonus awarded
3. HUD updates to show flow state
4. Flow streak timer begins (+1 bandwidth/minute)

---

7. Session Modes

7.1 Zen Mode

Philosophy: Gentle, supportive, forgiving

Aspect	Behavior
Penalty multiplier	0.5x - 0.75x
Intervention tone	Calming, supportive
Auto-dismiss	Yes (10s)
App blocking	Soft (can override)

7.2 Flow Mode (Default)

Philosophy: Balanced, practical, focused

Aspect	Behavior
Penalty multiplier	1.0x
Intervention tone	Clear, practical
Auto-dismiss	Yes (10s)
App blocking	Standard

7.3 Legend Mode

Philosophy: Aggressive, confrontational, unforgiving

Aspect	Behavior
Penalty multiplier	1.25x - 1.5x
Intervention tone	Harsh, direct
Auto-dismiss	**No**
App blocking	Hard (force close)

---

8. Parking Lot (Thought Capture)

8.1 Purpose

The **Parking Lot** is a quick-capture system for intrusive thoughts that would otherwise derail focus.

8.2 Item Types

Type	Description
Task	Something to do later
Idea	Creative thought to explore
Question	Research or ask someone
Follow-up	Need to circle back

8.3 Item Lifecycle

```
Capture → Review (Post-Session) → Action/Complete/Dismiss
```

1. **During session:** Quick add via HUD button
2. **Post-session harvest:** Review each item, categorize, plan
3. **Next session:** Carry forward or complete

8.4 Bandwidth Restoration

Adding items to the parking lot triggers a **Dump Reset** (+6 bandwidth) when multiple items are captured, clearing mental clutter.

---

9. Reset Rituals

9.1 Available Rituals

Ritual	Duration	Bandwidth Gain	Activity
Breath Reset	2 min	+5	Guided breathing exercise

Ritual	Duration	Bandwidth Gain	Activity
Walk Reset	5 min	+7.5	Physical movement break
Dump Reset	2-3 min	+6	Parking lot brain dump

9.2 Ritual Flow

1. User pauses session or intervention triggers pause
2. Ritual selection panel appears
3. User selects ritual
4. Countdown timer runs
5. On completion, bandwidth restored
6. Session resumes

---

10. Gamification & Virality

10.1 Badge System

45 unique badges across categories:

Category	Examples
Milestone	First Blood, Week Warrior, Centurion
Streak	3-Day, 7-Day, 30-Day, 100-Day
Performance	Perfect Session, Bandwidth Master, Zero Distractions
Mode-Specific	Zen Master, Flow Architect, Legend Survivor
Resilience	Comeback Kid, Phoenix Rising
Shame	Rage Quit, Distraction Disaster, The Struggle

10.2 Badge Rarity

Rarity	Unlock Rate	Examples
Common	~80%	First Blood, Day One
Uncommon	~50%	Week Warrior, Mode Explorer
Rare	~20%	Month Master, Bandwidth Master
Epic	~5%	Century Club, Legend Survivor
Legendary	~1%	Year Strong, Perfect Year
Shame	~30%	Walk of shame badges

10.3 Social Sharing

Twitter/X Integration:

```
const twitterUrl = `https://twitter.com/intent/tweet?
text=${text}&hashtags=${hashtags}`
await openUrl(twitterUrl) // Opens in default browser
```

LinkedIn Integration:

```
const linkedInUrl = `https://www.linkedin.com/sharing/share-offsite/?
url=${appUrl}`
await openUrl(linkedInUrl)
```

10.4 Streak System

Streak Type	Reset Condition
Daily	Miss a calendar day
Weekly	Miss a calendar week
Legend Daily	Miss a Legend mode day
Perfect Week	Any missed day in current week

---

11. Data Persistence

11.1 SQLite Database

All data is stored locally in SQLite:

```
~/.dustoff-reset/data.db
```

11.2 Data Models

Table	Purpose
`calibrations`	Daily calibration records
`sessions`	Session history and metrics
`parking_lot_items`	Thought capture items
`user_badges`	Unlocked badges
`streaks`	Streak records
`telemetry_events`	App switch/distraction events
`reflections`	Post-session reflections

11.3 Session Recovery

If app crashes during a session:

1. Session state is saved every 30 seconds
2. On next launch, recovery modal appears
3. User can resume or discard

---

12. Window Management

12.1 Window Characteristics

Property	Value
Decorations	None (frameless)
Transparency	Yes (glassmorphism)
Always on top	Yes (overlay)
Resizable	Yes (programmatic)
Skip taskbar	Yes

12.2 Dynamic Sizing

The window resizes based on current panel:

Panel	Dimensions
HUD only	320 × 80
Calibration	420 × 720
Pre-session	420 × 700
Intervention	520 × 720
Post-session	640 × 850

12.3 Window API

```
await tauriBridge.resizeWindow(width, height)
await tauriBridge.setWindowPosition(x, y)
await tauriBridge.startDragging() // Frameless drag
await tauriBridge.setAlwaysOnTop(true)
```

---

13. Permission System

13.1 Required Permissions

Permission	Platform	Purpose
Accessibility	macOS	Detect frontmost app, window titles
Screen Recording	macOS	Future: screenshot-based detection

13.2 Permission Check

```
// Uses AXIsProcessTrusted() via osascript
let output = Command::new("osascript")
    .arg("-l").arg("JavaScript")
    .arg("-e").arg("ObjC.import('ApplicationServices');
$.AXIsProcessTrusted()")
    .output()
```

13.3 Permission Flow

1. App launches → check permissions
2. If not granted → show permission setup panel
3. User opens System Settings → enables permission
4. User clicks "Check Again" → verify grant
5. If granted → hide panel, proceed normally

---

14. Cross-Platform Support

14.1 Current Status

Platform	Status
macOS	✅ Full support
Windows	🔄 In development
Linux	🔄 In development

14.2 Platform Abstraction

The codebase uses a **PlatformMonitor** trait for cross-platform compatibility:

```
pub trait PlatformMonitor {
    fn get_frontmost_app(&self) -> Result<ActiveAppInfo, String>;
    fn get_idle_time_seconds(&self) -> Result<u64, String>;
    fn platform_name(&self) -> &'static str;
}
```

macOS: Uses `objc` crate + AppleScript
Windows: Will use Win32 APIs (`GetForegroundWindow`)

Linux: Will use X11/Wayland APIs

Summary of Technical Capabilities

Capability	Implementation
Real-time app detection	Objective-C runtime (macOS)
Window title capture	AppleScript
Bandwidth calculation	Biologically-grounded algorithms
Flow state detection	4-condition threshold system
Intervention system	Mode-specific escalation
Session recovery	Auto-save + recovery modal
Gamification	45 badges, streaks, social sharing
Data persistence	Local SQLite database
Window management	Frameless overlay, dynamic sizing
Cross-platform	Trait-based abstraction

Future Capabilities (Roadmap)

Feature	Description
AI Focus Coach	Personalized recommendations
Team Analytics	Manager dashboard
Calendar Integration	Auto-block focus time
Browser Extension	Tab-level blocking
Mobile Companion	Break reminders, stats
Wearable Integration	HRV-based calibration

Document generated: January 2026
Dustoff Reset v0.1.0